Task 1: Bank Marketing with Structured Data – Experimental Report

Group Members:
Cheng Pengqi 225040121
Peng Qiheng 225040065
Liang Jiawen 225040105
Guo Xuanxuan 225040055
Xu Yiqing 225040049
Yi Xinru 225040061

MDS5020
School of Data Science, The Chinese University of Hong Kong, Shenzhen

Abstract

This report describes our solution to Task 1: Bank Marketing with Structured Data. The goal is to predict whether a client will subscribe to a term deposit based on historical bank marketing data. We design a preprocessing pipeline including recency-based transformation for `pdays`, handling of extreme values, missing-value imputation, categorical encoding and feature scaling. We implement several machine learning models, including Decision Tree, Logistic Regression, Naïve Bayes, K-Nearest Neighbors, Support Vector Machine and ensemble methods such as Random Forest, Gradient Boosting and a soft Voting classifier. Using 5-fold stratified cross-validation on the training data, Gradient Boosting Decision Trees (GBDT) achieve the best performance with a mean AUC of 0.7972. Following the project requirement, we use this best model to generate ranking scores for all 8,000 clients in `bank_marketing_test.csv`.

1 Data Understanding

The training dataset contains 26,246 observations with 25 variables related to direct marketing campaigns (telephone calls) of a Portuguese banking institution. The target variable y indicates whether the client subscribed to a term deposit (`yes` or `no`).

1.1 Variable Types and Basic Statistics

We first summarize the columns, data types, missing-value patterns and basic descriptive statistics for both numeric and categorical variables. The code and full tables are provided in the notebook `task1.ipynb`. Numeric summary statistics and correlations are visualized in the histogram, boxplot and correlation heatmap shown in Appendix A (Figures 2–5).

The features can be grouped as follows:

- **Client demographics**: `age`, `job`, `marital`, `education`, as well as credit-related flags `default`, `housing`, `loan`.
- **Campaign-related variables**: `contact`, `month`, `day_of_week`, `campaign` (number of contacts during this campaign), `pdays` (days since last contact from a previous campaign), `previous` (number of prior contacts), `poutcome` (outcome of previous campaign).
- **Macro-economic indicators**: `emp.var.rate`, `cons.price.idx`, `cons.conf.idx`, `euribor3m`, `nr.employed`.
- **Engineered features**: `feature_1`–`feature_5`.

1.2 Class Imbalance and Distributions

The positive class (`y = yes`) is much rarer than the negative class, leading to a noticeable class imbalance. This motivates the use of AUC as the primary evaluation metric and stratified sampling in cross-validation.

Using the numeric histograms and boxplots in Figures 2 and 3, we observe that many numeric variables are skewed and contain potential outliers, especially in `campaign`, `previous`, `emp.var.rate` and the engineered variables. Categorical distributions (Figure 4) show that some categories such as certain jobs and education levels are dominant, while others have very small support, which affects the choice of encoding.

The correlation heatmap in Figure 5 illustrates that several macro-economic indicators (e.g., `emp.var.rate`, `euribor3m`, `nr.employed`) are strongly correlated with each other, indicating potential redundancy and the need for regularization in linear models.

1.3 Special Values and Missingness

Two challenges are observed (see also `task1.ipynb`):

- **Special encoded values**: the value 999 in `pdays` denotes that the client was not previously contacted. The engineered variable `feature_1` contains $\pm\infty$, which breaks many standard algorithms if left untreated.

- **Missing values**: several variables, including `campaign`, `feature_2` and some macro-economic indicators, have missing entries. We treat these systematically in the preprocessing pipeline described in the next section.

2 Data Preparation and Feature Engineering

To obtain clean and homogeneous inputs for all models, we construct a preprocessing function applied to both training and test data. All statistics used for imputation and scaling are computed on the training set only to avoid data leakage.

2.1 Feature Transformation

Recency transformation for `pdays`. The value 999 in `pdays` denotes that the client was not contacted before the current campaign. To reflect the intuition that more recent contacts are more influential while 999 should correspond to low recency, we define

$$\text{pdays\_recency} = \frac{1}{\text{pdays} + 1}, \tag{1}$$

and drop the original `pdays` column.

Extreme values in `feature_1`. `feature_1` contains infinite values. We replace $\pm\infty$ with a large placeholder and create

$$\text{feature\_1\_transformed} = \frac{1}{\text{feature\_1}}, \tag{2}$$

which compresses extreme magnitudes. The original `feature_1` is then removed.

2.2 Handling Missing Values

Numeric variables (e.g., `age`, `campaign`, `previous`, `feature_2` and macro-economic indicators) are imputed using the mean of the training data. The ordinal variable `education` is first mapped to ordered integers and then imputed using the mode of the training set.

2.3 Encoding Categorical Variables

We use different encoders according to variable types:

- **Ordinal encoding**: `education` is mapped from `illiterate` (0) to `university.degree` (6), with `unknown` imputed as the most frequent level.
- **Binary mapping**: for `default`, `housing` and `loan`, we map `yes` $\to 1$, `no` $\to 0$, and `unknown` $\to -1$ to explicitly represent uncertainty.

- **One-hot encoding**: nominal variables including `job`, `marital`, `contact`, `month`, `day_of_week`, `poutcome`, `feature_3`, `feature_4` and `feature_5` are one-hot encoded with `drop_first=True`.

2.4 Feature Scaling

Since models such as Logistic Regression, KNN and SVM are sensitive to feature scales, we standardize all numeric features (original numeric variables plus `pdays_recency`, `feature_1_transformed` and the numeric `education`) using `StandardScaler` so that each has zero mean and unit variance.

3 Methodology

The task is a binary classification problem evaluated by AUC. We follow the project guideline and compare several classical models as well as ensemble methods.

3.1 Base Models

1. **Decision Tree (DT)**: a tree-based non-linear model serving as a simple baseline.
2. **Logistic Regression (LR)**: a linear classifier well suited for high-dimensional sparse features.
3. **Naïve Bayes (NB)**: Gaussian Naïve Bayes assuming conditional independence of features.
4. **K-Nearest Neighbors (KNN)**: a non-parametric classifier with $k = 5$ neighbors.
5. **Support Vector Machine (SVM)**: RBF-kernel SVM with `probability=True` to enable probability outputs and AUC computation.

3.2 Ensemble Models

1. **Random Forest (RF)**: a bagging ensemble of 100 decision trees to reduce variance.
2. **Gradient Boosting Decision Trees (GBDT)**: a boosting ensemble with 100 weak learners, trained sequentially to correct previous residual errors.
3. **Soft Voting Classifier**: an ensemble that combines DT, LR, NB, KNN and SVM by averaging their predicted probabilities.

4 Experiments and Results

4.1 Experimental Setting

We use the provided training set (26,246 samples) for model selection and evaluation. To obtain robust estimates under class imbalance, we adopt 5-fold **stratified** cross-validation. In each fold, four folds are used for training and the remaining fold for validation. We record both accuracy and AUC, but report AUC as the main metric, consistent with the Task 1 description.

Implementation details, including the preprocessing code, model initialization and training loop, are contained in `task1.ipynb`.

4.2 Performance Results

The performance of each model is summarized in Table 1. The values are the mean AUC scores across the 5 folds.

表 1: Model performance (5-fold stratified cross-validation).

Model Type	**Algorithm**	**Mean AUC (5-Fold)**
Decision Tree	DecisionTreeClassifier	0.6205
Logistic Regression	LogisticRegression	0.7918
Naïve Bayes	GaussianNB	0.7253
K-Nearest Neighbors	KNeighborsClassifier	0.7215
Support Vector Machine	SVC (RBF, probability=True)	0.7071
Random Forest (RF)	RandomForestClassifier	0.7687
GBDT	GradientBoostingClassifier	**0.7972**
Soft Voting Ensemble	VotingClassifier (soft)	0.7680

GBDT achieves the highest mean AUC of 0.7972 and is therefore selected as our final model for Task 1. As required in the project description, we use the trained GBDT model on the full training data to produce ranking scores for all 8,000 clients in `bank_marketing_test.csv`. The predicted probabilities of y = yes are stored in `output/GBDT_test_scores.csv`, which matches the expected format of 8,000 rows and one column.

4.3 Discussion

Tree-based ensemble methods (RF and GBDT) clearly outperform the single Decision Tree baseline, confirming the benefit of ensembling on this structured dataset. GBDT slightly but consistently surpasses Logistic Regression, suggesting that non-linear interactions between features are useful.

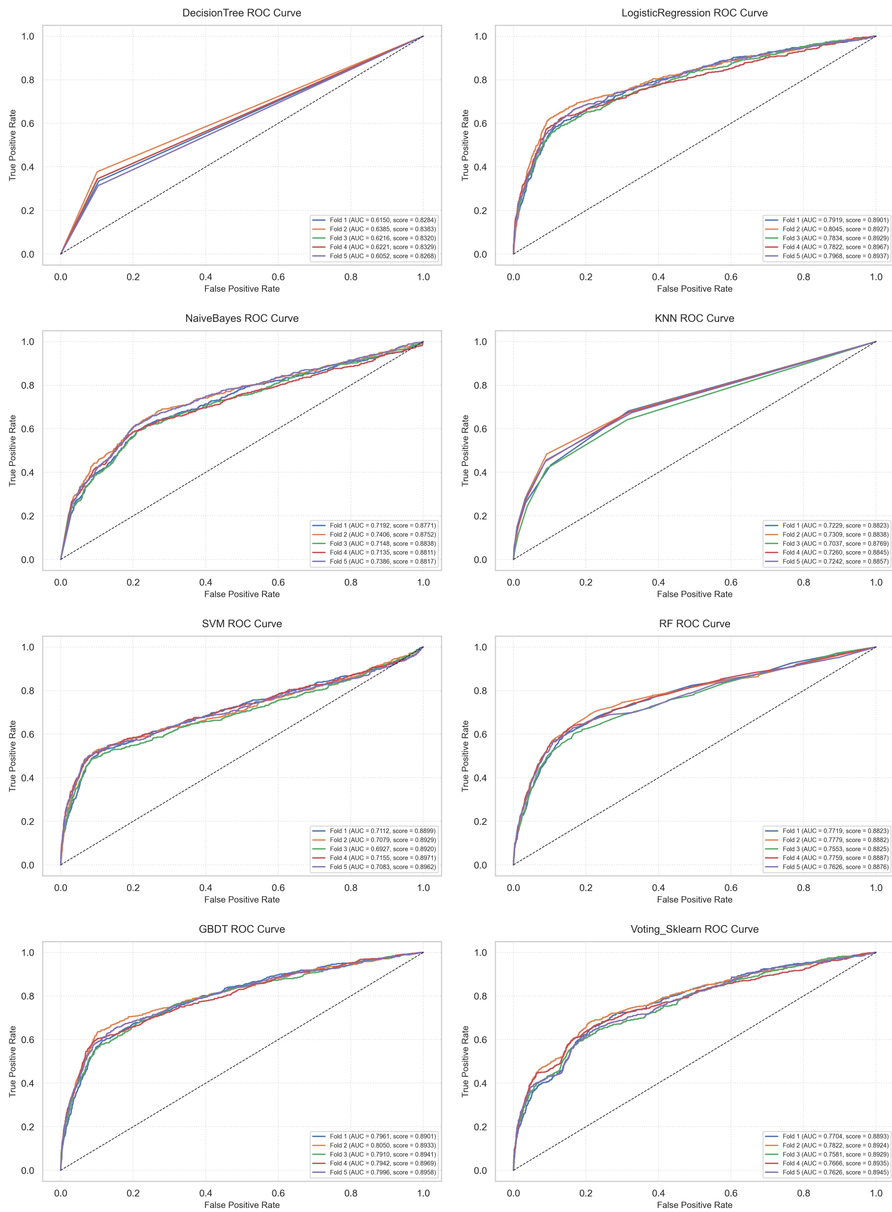

图 1: ROC curves of different models across the 5 validation folds.

KNN and SVM are less competitive, likely because of the high dimensionality induced by one-hot encoding and the limited hyperparameter tuning. The soft Voting ensemble does not exceed the best single model; naive averaging tends to dilute the strong signal from GBDT together with weaker learners.

5 Conclusion

We built a complete pipeline for Task 1 of the MDS5020 group project, from data preprocessing and feature engineering to model comparison and selection. Using 5-

fold stratified cross-validation, we identified GBDT as the best-performing model with a mean AUC of 0.7972. Following the assignment requirement, we used this model to generate the ranking scores file `bank_marketing_test_scores.csv`, which can be evaluated by the instructor against the hidden test labels.

A Additional Data Understanding Figures

This appendix contains the figures generated in `task1.ipynb` that support the data understanding section.

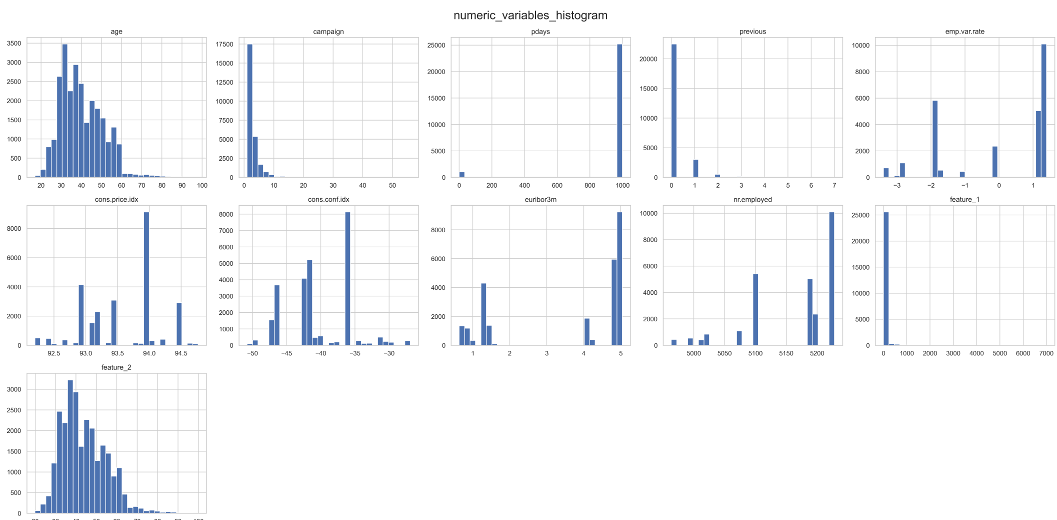

图 2: Histograms of numeric variables in the training set.

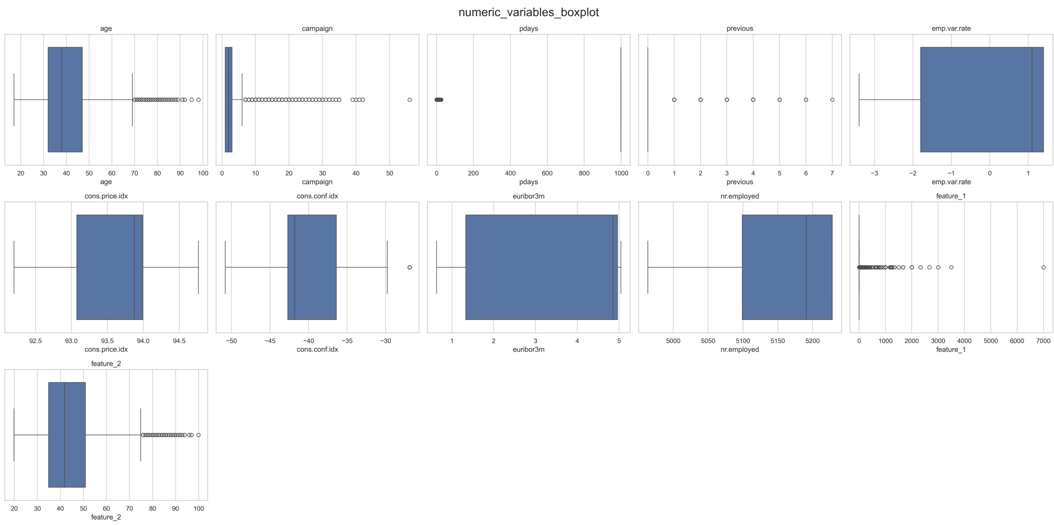

图 3: Boxplots of numeric variables in the training set.

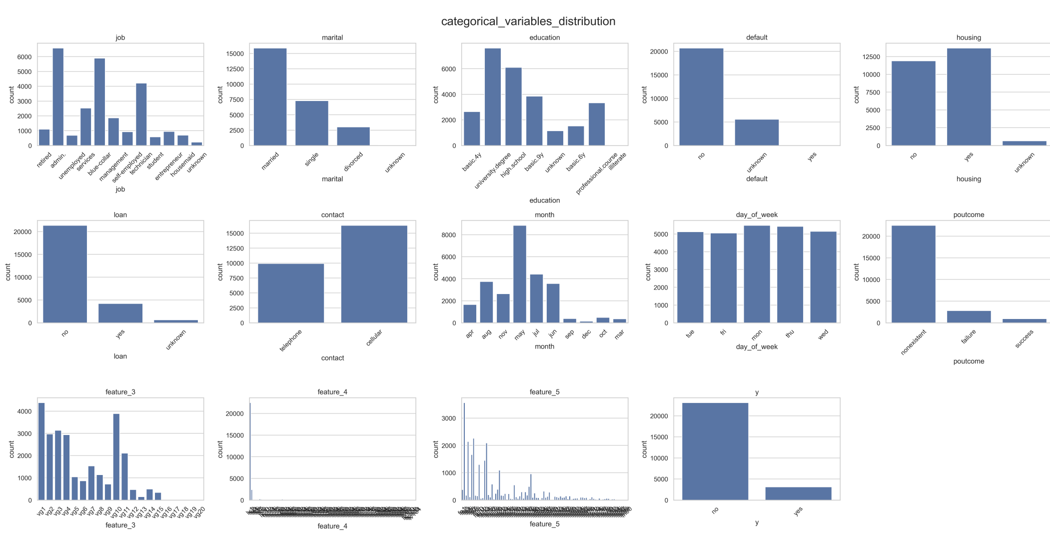

图 4: Frequency distributions of categorical variables in the training set.

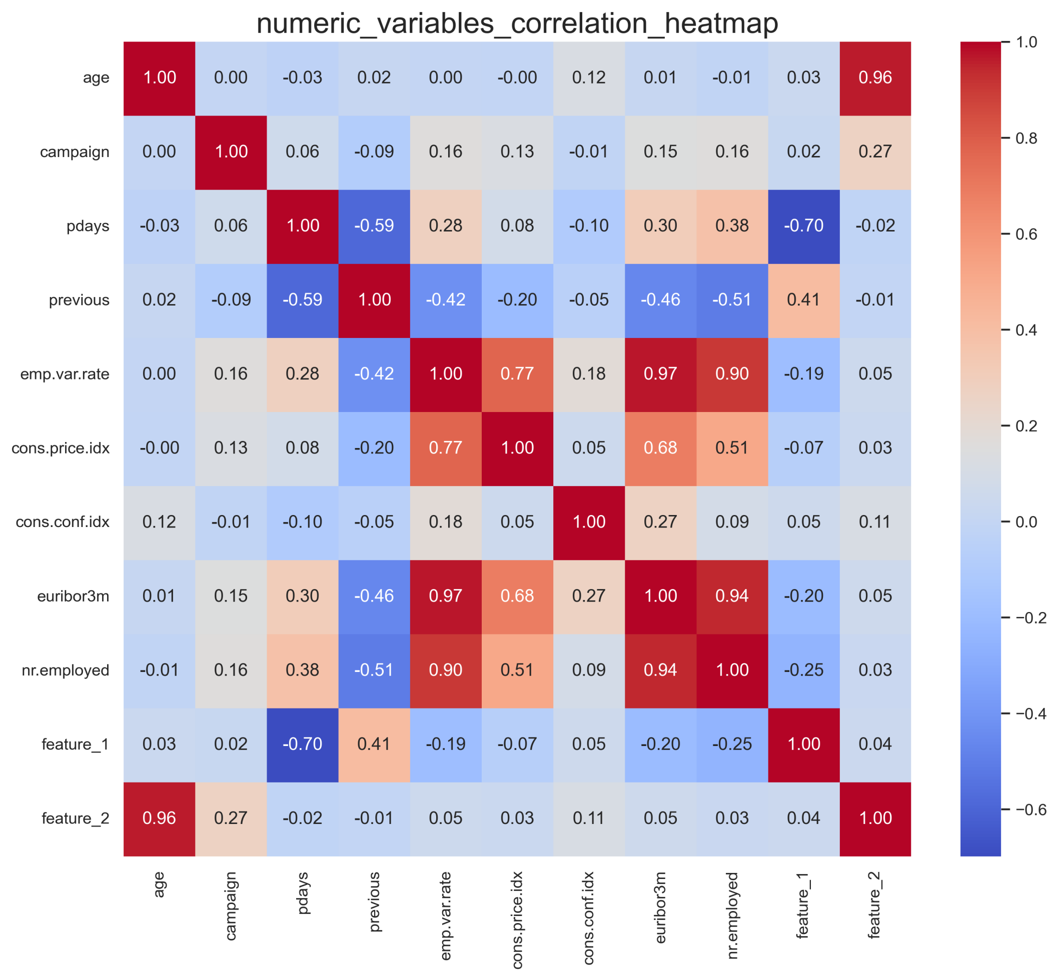

图 5: Correlation heatmap of numeric variables in the training set.